ClimateSense Odisha — Complete Project Report

Project: District-wise climate analysis and machine-learning prediction for Odisha Data source: NASA POWER daily local CSV exports Report date: 29 October 2026

1. Executive summary

ClimateSense Odisha is a reproducible Python pipeline that discovers, validates, cleans, explores, splits, trains, evaluates, and predicts district-wise climate variables. The project currently uses 15 NASA POWER district CSV files containing 136,965 daily records from 1 January 2001 to 31 December 2025.

The four required prediction targets are mean temperature (`T2M`), corrected precipitation (`PRECTOTCORR`), relative humidity (`RH2M`), and wind speed (`WS2M`). The output units are degrees Celsius, mm/day, percent, and m/s respectively, according to the NASA POWER metadata present in the local files.

The project has two separate products. Historical test evaluation measures performance on a held-out chronological test period. Future prediction generates one row per district. Since the local source currently ends on 31 December 2025, a prediction generated on 29 October 2026 is correctly marked as a historical same-calendar-day climatological estimate rather than being presented as a live weather forecast.

2. Objectives completed

• Dataset discovery and inventory.

• NASA POWER metadata and unit inspection.

• Raw-data preservation.

• Data cleaning and date construction.

• Missing-value and duplicate checks.

• Exploratory data analysis and climate summaries.

• Leakage-safe feature engineering.

• Chronological train/validation/test splitting.

• Separate models for temperature, rainfall, humidity, and wind speed.

• Held-out model evaluation.

• Model persistence and metadata.

• District-level next-day CSV output.

• Interactive console prediction.

• Automated tests and project documentation.

The project is currently a command-line research/college-lab pipeline. It has no web application, API, dashboard, database, scheduler, or live data connector.

3. Dataset inventory

The raw directory contains these 15 districts: Angul, Bargarh, Bhadrak, Cuttack, Gajapati, Ganjam, Kalahandi, Kendrapada, Keojhar, Khurda, Koraput, Malkangiri, Puri, Sambalpur, and Sundargarh.

Property	Actual value
Number of files	15
Total rows	136,965
Rows per file	9,131
Start date	2001-01-01
End date	2025-12-31
Time basis	LST/local solar time
NASA missing sentinel	-999

Each source file represents one district. The combined dataset preserves the district and source-file identity. Original CSV files in `data/raw/` were not modified.

4. Variables and verified units

Column	Meaning	Unit	Project role
`YEAR`	Calendar year	year	Date construction
`DOY`	Day of year	day number	Date construction/seasonality
`T2M`	Mean air temperature at 2 m	°C	Primary target
`T2M_MAX`	Maximum temperature at 2 m	°C	Input/additional target
`T2M_MIN`	Minimum temperature at 2 m	°C	Input/additional target
`T2MDEW`	Dew/frost-point temperature	°C	Input feature
`RH2M`	Relative humidity at 2 m	%	Primary target
`PRECTOTCORR`	Corrected precipitation	mm/day	Primary target
`WS2M`	Wind speed at 2 m	m/s	Primary target
`ALLSKY_SFC_SW_DWN`	Downward shortwave radiation	MJ/m²/day	Input feature
`PS`	Surface pressure	kPa	Input feature

The project does not substitute maximum/minimum temperature for mean temperature, or dew point for humidity. Rainfall remains a numeric amount and is not confused with a probability of rain.

5. Directory and module design

- `data/raw/`: original NASA POWER inputs.
- `data/processed/`: cleaned analysis-ready data.
- `data/splits/`: chronological split files.
- `src/climatesense/`: reusable loading, validation, preprocessing, analysis, and feature functions.
- `scripts/`: executable pipeline stages.
- `models/multi_target/`: target-specific joblib models and JSON metadata.
- `outputs/reports/`: inventories, quality reports, split reports, and findings.
- `outputs/figures/`: EDA and evaluation figures.
- `outputs/metrics/`: metrics, test predictions, and next-day predictions.
- `tests/`: automated tests.
- `docs/`: project documentation.

6. Data loading and preprocessing

The loader discovers supported files instead of assuming a fixed filename. It handles NASA metadata/header rows, normalizes column names, preserves source provenance, and records file-level audit information.

The preprocessing stage:

1. Reads the raw CSVs.
2. Parses `YEAR` and `DOY` into valid dates.
3. Validates leap years and day-of-year bounds.
4. Converts climate columns to numeric values.
5. Converts NASA missing-value sentinels to missing values rather than zero.
6. Normalizes district names.
7. Sorts records by district and date.
8. Checks duplicates, missing values, invalid dates, and suspicious measurements.
9. Saves `data/processed/climate_daily_cleaned.csv`.

The preprocessing run saved 136,965 cleaned rows. Raw files are never overwritten. Zero rainfall is retained as a real observation; missing rainfall is not silently replaced with zero.

7. Exploratory data analysis

The EDA pipeline uses Pandas, NumPy, Matplotlib, and Seaborn. It generates dataset summaries, type and missingness tables, distributions, boxplots, time-series plots, seasonal summaries, district comparisons, correlation tables/heatmaps, rainfall analyses, and trend summaries where supported.

The reports are saved in `outputs/reports/` and plots in `outputs/figures/`. Correlation is treated as association, not proof of causation. Long-term observed trends are kept separate from short-term predictions.

8. Feature engineering and leakage prevention

For each target and district, the model-ready data contains:

• 1-day, 2-day, 3-day, 7-day, and 14-day lags.
• A shifted 7-day rolling mean.
• Month and day of year.
• Sine and cosine seasonal encodings.
• District identifier.

All features are calculated after sorting by district and date. The rolling window is shifted before aggregation, so the target day's observed value is not included in its own features. Inference uses only observations available on or before the reference date. The exact feature names and ordering are stored with the model artifact.

9. Chronological splitting

The configured split is approximately 70% training, 15% validation, and 15% testing:

Split	Rows	Date range
Training	95,865	2001-01-01 to 2018-07-01
Validation	20,550	2018-07-02 to 2022-04-01
Test	20,550	2022-04-02 to 2025-12-31

Time-series rows are not randomly shuffled. The test period is not used for model fitting or model selection. District counts and overlap checks are written to `outputs/reports/split_summary.json`. Lagged model-ready counts are slightly smaller because the first days of each district cannot supply all lag values.

10. Algorithms

HistGradientBoostingRegressor

Temperature, humidity, and wind-speed models use scikit-learn's `HistGradientBoostingRegressor`. It builds an additive sequence of decision trees, where later trees correct earlier errors. Histogram binning provides efficient training on large tabular data. Main parameters are `max_iter=180`, `learning_rate=0.06`, and `random_state=42`.

Rainfall occurrence model

Rainfall is intermittent and contains many zero values. The first rainfall model is a `HistGradientBoostingClassifier` that predicts the binary event `PRECTOTCORR > 0`. Its thresholded result determines whether the predicted amount is zero or positive.

Rainfall amount model

For positive rainfall rows, a second `HistGradientBoostingRegressor` predicts `log1p(PRECTOTCORR)`. The prediction is transformed back using `expm1` and clipped at zero. This two-stage approach handles rainfall's zero-inflated and skewed distribution more appropriately than one ordinary regression model.

Preprocessing components

The model pipelines use `OneHotEncoder` for districts, `SimpleImputer(strategy="median")` for numeric features, `ColumnTransformer` for separate categorical/numeric processing, and scikit-learn `Pipeline` objects. Learned transformations are fitted on training data only and applied to validation/test data.

11. Actual evaluation results

The following values are from the chronological test stage in `outputs/metrics/multi_target_metrics.csv`:

Target	MAE	RMSE	R^2	Rainfall/event metric
T2M	0.5571 °C	0.7649 °C	0.9732	—
RH2M	3.1832%	4.4465%	0.9369	—
WS2M	0.4122 m/s	0.5735 m/s	0.6999	—
PRECTOTCORR	2.7789 mm/day	6.6192 mm/day	Not used	F1 0.9410; ROC-AUC 0.9672

The separate final baseline temperature test artifact reports MAE `0.1899 °C`, RMSE `0.2487 °C`, R^2 `0.9972`, mean bias `-0.0097 °C`, and maximum absolute error `1.3492 °C`.

These are historical held-out results. They do not guarantee future operational forecast performance.

12. Saved model artifacts

The four primary artifacts are `models/multi_target/T2M.joblib`, `PRECTOTCORR.joblib`, `RH2M.joblib`, and `WS2M.joblib`. Additional `T2M_MAX` and `T2M_MIN` artifacts are also available.

Each metadata JSON records target, unit, feature columns, model type, parameters, train/validation/test ranges and counts, data fingerprint, model version, and creation time. This protects inference from silently using an incompatible feature schema.

13. Next-day prediction behaviour

`predict_tomorrow.py` calculates the next calendar date, finds the latest observation for each district, and measures the data gap.

- For data no older than seven days, it uses target-specific lag models.
- For data older than seven days, it calculates a same-calendar-day historical mean and sets status to `climatological_estimate`.
- Missing models or missing features produce null values and explicit unavailable statuses, never fabricated numbers.

The wide output contains district, last observed date, forecast date, four numeric prediction columns, four status columns, four model-name columns, generation time, data gap, data source, and data-quality status.

Because the current source ends on 31 December 2025, a forecast for 30 October 2026 is a stale-source historical estimate. To obtain a true model-based next-day forecast, add current data and rerun preprocessing, splitting, training, and prediction.

14. Commands from beginning to end

Run from the project root in PowerShell:

```
.\.venv\Scripts\Activate.ps1

python scripts/inspect_datasets.py
python scripts/run_preprocessing.py
python scripts/run_eda.py
python scripts/validate_data_freshness.py
python scripts/prepare_splits.py

python scripts/run_training.py
python scripts/run_multi_target_training.py

python scripts/generate_final_predictions.py
python scripts/predict_tomorrow.py
python scripts/format_prediction_outputs.py

python -m pytest
```

`generate_final_predictions.py` evaluates the historical held-out test period. `predict_tomorrow.py` creates the future district output. `format_prediction_outputs.py` creates human-readable display fields.

15. Interactive prediction commands

Run:

```
python scripts/interactive_next_day_prediction.py
```

Example input:

```
Enter reference date (YYYY-MM-DD): 2026-10-29
Enter district name: gajapati
```

The script prints the predicted temperature, rainfall, humidity, wind speed, units, prediction status, forecast date, latest observation, data gap, model names, and data-quality warning when the source is stale.

16. Output files

Important outputs are `outputs/metrics/final_test_predictions.csv`, `final_test_metrics.json`, `multi_target_metrics.csv`, `multi_target_metrics.json`, `tomorrow_predictions_current.csv`, `tomorrow_predictions_display.csv`, and `tomorrow_prediction_summary.json`.

The raw CSV values remain numeric. The display CSV formats temperature to one decimal place, rainfall to two decimals, humidity to one decimal place, and wind speed to one decimal place. If a CSV is open in Excel or VS Code, Windows may force the script to create a timestamped replacement.

17. Libraries and packages

• Python standard library: `pathlib` for paths, `datetime` for dates, `json` for reports, `hashlib` for fingerprints, and `sys` for imports.

• Pandas: CSV loading, grouping, date parsing, cleaning, summaries, and exports.

• NumPy: numerical arrays, seasonal trigonometric features, clipping, and transformations.

• scikit-learn: pipelines, encoders, imputers, gradient boosting, and metrics.

• joblib: saving and reloading trained models.

• Matplotlib: figures and charts.

• Seaborn: statistical plots and styling.

• SciPy/statsmodels: supported statistical analysis modules.

• PyYAML: YAML configuration support.

• Jupyter/ipykernel: notebook execution support.

• pytest: automated tests.

18. Testing

The current automated suite covers final artifacts, formatting, core pipeline behaviour, and the four-variable tomorrow-prediction schema. The latest completed run reported:

```
7 passed
```

The scripts are rerunnable, raw files are preserved, and generated files are kept separate from source data.

19. Limitations

1. The local data ends on 31 December 2025, so 2026 future output is stale-source protected.
2. No live NASA POWER API refresh is implemented.
3. Historical lag models cannot know unobserved future atmospheric conditions without recent observations or another forecast source.
4. Rainfall is intermittent and highly skewed; the two-stage approach does not remove uncertainty.
5. Historical test metrics are not guarantees of operational forecast accuracy.
6. This is a research and college-lab command-line pipeline, not a production weather service.

20. Conclusion

ClimateSense Odisha now provides a complete reproducible workflow from district-wise NASA POWER CSV files to cleaned data, EDA, chronological splits, target-specific models, held-out evaluation, and four-variable district-level next-day output. Data freshness is the key operational requirement. Add current observations and rerun the complete workflow before treating the output as a genuine next-day model forecast.